

EXAMEN DE PROGRAMACIÓN – 1º GRADO SUPERIOR

Examen 5 · Ficheros, excepciones y colecciones

Lectura y escritura de ficheros de texto (Scanner, File, PrintWriter)

Gestión de excepciones: try/catch, finally, try-with-resources

Excepciones comprobadas y no comprobadas. jerarquía Throwable

Colecciones: ArrayList, HashMap, HashSet

Uso combinado de ficheros y colecciones

Asignatura	Programación (PRG)
Curso	1º Grado Superior — Desarrollo de Aplicaciones Web (DAW)
Duración	2 horas
Puntuación total	16 puntos
Convocatoria	Evaluación 2
Valoración	Ficheros, excepciones y colecciones

Parte 1: Opción múltiple y Verdadero/Falso

5 puntos — 0,5 cada pregunta

1. ¿Qué bloque se ejecuta SIEMPRE, haya o no excepción?
- a) try
 - b) catch
 - c) finally
 - d) throws

RESPUESTA / SOLUCIÓN

c) finally — El bloque finally se ejecuta en todos los casos (con o sin excepción), salvo `System.exit()`.

-
2. ¿Qué clase es la base de todas las excepciones en Java?
- a) Error
 - b) Exception
 - c) Throwable
 - d) RuntimeException

RESPUESTA / SOLUCIÓN

c) Throwable — Es la raíz de la jerarquía; de él derivan Exception y Error.

-
3. Una excepción comprobada (checked) obliga a:
- a) Capturarla o declararla con throws
 - b) Ignorarla

- c) Lanzarla siempre
- d) Convertirla a RuntimeException

RESPUESTA / SOLUCIÓN

a) Las excepciones comprobadas deben capturarse (try/catch) o declararse en la firma con throws.

-
4. La sentencia try-with-resources cierra automáticamente los recursos definidos dentro de ella.

RESPUESTA / SOLUCIÓN

Verdadero — try-with-resources (Java 7+) cierra automáticamente los recursos AutoCloseable al final del bloque, sin necesidad de finally.

-
5. La clase `PrintWriter` se usa principalmente para:

- a) Leer de un fichero
- b) Escribir texto formateado en un fichero de salida
- c) Crear un directorio
- d) Borrar un fichero

RESPUESTA / SOLUCIÓN

b) PrintWriter escribe texto, a menudo con formato (printf/println), en un archivo de salida.

6. ¿Qué estructura permite almacenar pares clave-valor y no admite claves duplicadas?

- a) ArrayList
- b) HashSet
- c) HashMap
- d) LinkedList

RESPUESTA / SOLUCIÓN

c) HashMap — Almacena pares clave-valor; las claves son únicas (si se inserta una clave existente, se reemplaza el valor).

7. ¿Qué imprime este código?

```
List<Integer> l = new ArrayList<>();  
l.add(10); l.add(20); l.add(10);  
System.out.println(l.size());
```

- a) 2
- b) 3
- c) 4
- d) Error

RESPUESTA / SOLUCIÓN

b) 3 — ArrayList permite elementos duplicados, por lo que hay 3 elementos pese a repetir el 10.

8. Un HashSet garantiza el orden de los elementos en la inserción.

RESPUESTA / SOLUCIÓN

Falso — HashSet no garantiza ningún orden de los elementos; para orden de inserción se usa LinkedHashSet o List.

9. ¿Qué ocurre al intentar leer un fichero que no existe?

- a) Se crea automáticamente
- b) Se lanza FileNotFoundException
- c) Devuelve un fichero vacío
- d) Se lanza NumberFormatException

RESPUESTA / SOLUCIÓN

b) FileNotFoundException (subclase de IOException, comprobada) se lanza cuando se intenta abrir un fichero inexistente para lectura.

10. Para lanzar una excepción propia dentro de un método se usa la palabra clave:

- a) throw
- b) throws
- c) try
- d) new

RESPUESTA / SOLUCIÓN

a) throw lanza una excepción; `throws` declara las excepciones que un método puede propagar.

Parte 2: Análisis y depuración

3 puntos

1. El siguiente código tiene errores. Encuéntralos y corrígelos. (1,5 ptos)

```
import java.io.*;
public class LeerArchivo {
    public static void main(String[] args) {
        try {
            BufferedReader br = new BufferedReader(new FileReader("datos.txt"));
            String linea;
            while ((linea = br.readLine()) != null) {
                System.out.println(linea);
            }
        }
        System.out.println("Fin");
    }
}
```

RESPUESTA / SOLUCIÓN

1. No se captura la `IOException`, que el compilador exige (checked). Falta `catch (IOException e) { ... }`.
2. El `BufferedReader` no se cierra. Mejor usar `try-with-resources`: `try (BufferedReader br = new BufferedReader(new FileReader("datos.txt")))`.
3. (opcional de diseño) Acepta también declarar `throws IOException` en `main`, aunque lo preferible es capturar la excepción.

2. Determina la salida de este código (traza con excepciones). (1,5 ptos)

```
public class ExpTraza {
    static int dividir(int a, int b) {
        try { return a / b; }
        catch (ArithmeticException e) { System.out.println("Arithmetic"); return -1; }
        finally { System.out.println("finally"); }
    }
    public static void main(String[] a) {
        System.out.println(dividir(10, 2));
        System.out.println(dividir(10, 0));
    }
}
```

RESPUESTA / SOLUCIÓN

Primera llamada (10/2): no hay excepción; se ejecuta finally (imprime 'finally') y devuelve 5 → imprime 5.

Segunda llamada (10/0): se lanza ArithmeticException, se captura (imprime 'Arithmetic'), luego finally (imprime 'finally'), devuelve -1 → imprime -1.

Salida completa: ``finally 5 Arithmetic finally -1``.

Parte 3: Ejercicios de programación

7 puntos

1. Escribe un programa que lea un archivo `alumnos.txt` donde cada línea contiene `nombre nota1 nota2 nota3` separados por espacio. Calcula la media de cada alumno y escribe en `resultados.txt` el nombre y la media con 2 decimales. Gestiona las excepciones con try-with-resources. (2,5 pts)

RESPUESTA / SOLUCIÓN

```
import java.io.*;
import java.util.Scanner;

public class GestionAlumnos {

    public static void main(String[] args) {

        try (Scanner sc = new Scanner(new File("alumnos.txt"));
            PrintWriter pw = new PrintWriter("resultados.txt")) {

            while (sc.hasNextLine()) {

                String linea = sc.nextLine().trim();

                if (linea.isEmpty()) continue;

                String[] p = linea.split(" ");

                double s = 0; int n = p.length - 1;

                for (int i = 1; i < p.length; i++) s += Double.parseDouble(p[i]);

                pw.printf("%s: %.2f%n", p[0], s / n);

            }

            System.out.println("resultados.txt generado");

        } catch (FileNotFoundException e) {

            System.out.println("No se encuentra alumnos.txt");

        } catch (Exception e) {

            System.out.println("Error: " + e.getMessage());

        }

    }

}
```

2. Crea un programa que use un HashMap para guardar el número de habitantes de varias ciudades (clave=ciudad, valor=habitantes). Pide ciudades por teclado, inserta varias, y luego: recorre el mapa mostrando cada ciudad y su población, y muestra la ciudad con más habitantes. (2,5 pts)

RESPUESTA / SOLUCIÓN

```
import java.util.*;

public class Ciudades {

    public static void main(String[] args) {

        Map<String, Integer> hab = new HashMap<>();

        hab.put("Madrid", 3200000);

        hab.put("Barcelona", 1600000);

        hab.put("Valencia", 790000);

        hab.put("Madrid", 3300000); // reemplaza

        String mayor = null; int max = -1;

        for (Map.Entry<String,Integer> e : hab.entrySet()) {

            System.out.println(e.getKey() + " -> " + e.getValue());

            if (e.getValue() > max) { max = e.getValue(); mayor = e.getKey(); }

        }

        System.out.println("Más poblada: " + mayor);

    }

}
```

3. Escribe un método que valide si un número de hora es correcto (entre 0 y 23) y lance una excepción personalizada `HorasInvalidaException` si no lo es. Crea la excepción y un programa que pida una hora y la muestre, capturando la excepción. (2 pts)

RESPUESTA / SOLUCIÓN

```
class HoraInvalidaException extends Exception {  
    public HoraInvalidaException(String msg) { super(msg); }  
}  
  
import java.util.Scanner;  
  
public class Hora {  
    static void validar(int h) throws HoraInvalidaException {  
        if (h < 0 || h > 23) throw new HoraInvalidaException("Hora fuera de rango: " + h);  
    }  
  
    public static void main(String[] a) {  
        Scanner sc = new Scanner(System.in);  
        System.out.print("Hora: ");  
        int h = sc.nextInt();  
        try { validar(h); System.out.println("Hora válida: " + h); }  
        catch (HoraInvalidaException e) { System.out.println(e.getMessage()); }  
    }  
}
```

Parte 4: Pregunta teórica

1 punto

1. Explica la diferencia entre ArrayList y HashMap, y entre ArrayList y HashSet. ¿Cuándo usarías cada una?

RESPUESTA / SOLUCIÓN

ArrayList: lista ordenada indexada, admite duplicados, acceso por posición. Ideal para una secuencia ordenada o cuando importa el índice/orden.

HashMap: almacena pares clave→valor, claves únicas, sin orden garantizado, acceso por clave en $O(1)$. Ideal para búsquedas por clave (diccionarios).

HashSet: colección de elementos únicos, sin orden, no permite duplicados, operaciones de pertenencia eficientes. Ideal para eliminar duplicados o comprobar presencia.

Regla: necesitas orden/posición → ArrayList; clave→valor → HashMap; unicidad/presencia → HashSet.

Plantilla de respuestas

(Páginas en blanco para desarrollar las soluciones)